

Process – Fork Examples:

1.

```
main()
{
printf("Welcome.\n");
fork();//creates child process
printf("PID:%d, PPID:%d\n",getpid(), getppid()); //this is executed twice
while(1);
}
```

2.

```
#include<stdio.h>
#include<unistd.h>
main()
{
printf("Welcome.\n");
fork();
fork();
fork();
printf("PID:%d, PPID:%d\n",getpid(), getppid());
while(1);
}
```

3.

```
main()
{
printf("Welcome to Moschip\n");
if(fork())
{
printf("parent..\n");
```

```

}
else
{
printf("child...\n");
}
printf("both...\n");
}

```

4.

```

#include<stdio.h>
#include<unistd.h>

main()
{
int i;
printf("Welcome to Moschip\n");
for(i=0;i<3;i++)
{
if(fork()==0)
printf("i%d...pid%d...ppid%d\n",i,getpid(),getppid());
else
;
}
while(1);
}

```

5.

```

#include<stdio.h>

void main(void)
{
    if(fork()==0)

```

```

{
    printf("in child before sleep pid =%d ppid =%d n", getpid(),getppid());
    sleep(10);
} else {
    printf("in parent pid =%d ppid =%d n", getpid(), getppid());
    sleep(5);
}
}

```

6.

```

#include<stdio.h>

void main(void)
{
    if(fork()==0) { //child is executing for 10 seconds only
        printf("in child pid =%d ppid =%d n", getpid(), getppid());
        sleep(10);
        printf("Child is terminating n");
    }
    else { //parent is executing infinitely
        printf("in parent pid =%d ppid =%d n", getpid(), getppid());
        while(1);
    }
}

```

7.

```

#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>
int main()
{
    int i=5;
    int ret=fork();
    if(ret==-1)
    {
        perror("fork");
    }
}

```

```

        exit(0);
    }
    if(ret>0)
    {
        i+=10;
        printf("i=%d    %d\n",i,&i);

    }
    else
    {
        i=i+30;
        printf("i=%d %d\n",i,&i);
    }
}

```

8.

```

#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>
int main()
{
    int ret=fork();
    if(ret==-1)
    {
        perror("fork");
        exit(0);
    }
    if(ret>0)
    { //exclusively in parent process
        int ret1=fork();
        if(ret1==-1)
        {
            perror("child2");
            exit(0);
        }
        if(ret1==0)
        {
            printf("child2 PID=%d PPID=%d\n",getpid(),getppid());
        }
        else
        { //exclusive parent code
            sleep(20);
            printf("Parent process PID=%d  \n",getpid());
        }
    }
    else
    { //exclusively in child1 process
        sleep(10);
        printf("child1 PID=%d PPID= %d\n",getpid(),getppid());
    }
}

```

9.

```

#include <stdio.h>

#include <unistd.h>

```

```

int main()
{
    fork();

    fork() && fork() || fork();

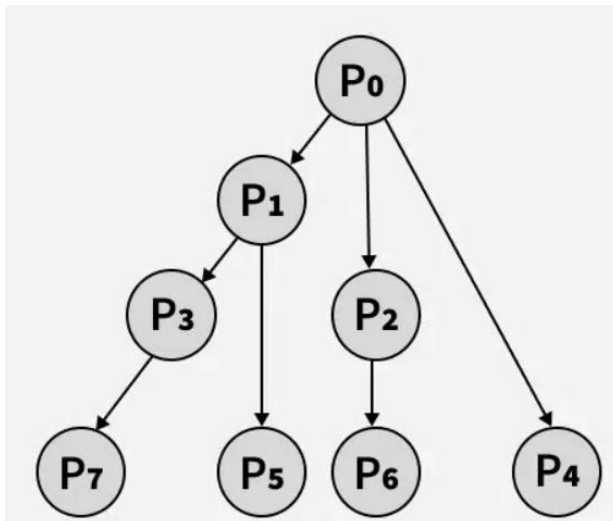
    fork();

    printf("forked\n");

    return 0;
}

```

10. Create an example for following.



11.

```

#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main()
{
    fork();

    fork();

    fork();

    printf("hello\n");
}

```

```
    return 0;  
}
```

Check from which process hello is printing,